



ARTIFICIAL INTELLIGENCE

PROGRAMMING PROJECT

May 3, 2026

1 Assignment Description

In this assignment, you are going to study in detail one AI algorithm of your choice. You are also asked to propose your own variations of the algorithm, and study the impact experimentally. Finally, you must write a report to document your findings. Hence, this is a very flexible and personalised study, which you can use to build your portfolio.

Note that your variations do not need to lead to better experimental results than the original algorithm. Your grade will be based on the quality of your work, not the numerical difference in the results between the original approach and the variations studied.

You are free to choose which algorithm you want to study. It does not have to be an algorithm that we have discussed in class.

You will need one (or more) environment(s) to evaluate the algorithm and your variations. There are several frameworks available that can be used, and you can choose which one you prefer. You can find below some suggestions:

- Gymnasium (<https://gymnasium.farama.org/index.html>): Contains several interesting environments for Reinforcement Learning, including Atari games. A fork from the original framework by OpenAI.
- OpenSpiel (<https://openspiel.readthedocs.io/en/latest/intro.html>): Several games for reinforcement learning and planning, by DeepMind.
- Neural MMO (https://neuralmmo.github.io/_build/html/rst/landing.html): A Massively Multiplayer Online Role-Playing Game (MMORPG) environment for AI agents.
- MAgent 2 (<https://magent2.farama.org/index.html>): Pixel agents interact in battles in a grid-world.
- Fighting ICE (<https://www.ice.ci.ritsumeai.ac.jp/~ftgaic/>): A 2D fighting game (i.e., like “Street Fighter”).

- Legend of Code and Magic (<https://github.com/acatai/Strategy-Card-Game-AI-Competition>): A card game battle game.
- AdLeap-MAS (<https://github.com/lsmcolab/adleap-mas>): Environments for studying ad-hoc teamwork (when an agent learns about other agents to decide how to work together), such as collaborative collection of items (level-based foraging). Developed in my lab.

Note that these are only suggestions, and you can use frameworks that are not in this list, as well as build your own environment. Similarly, you can do your project in the programming language of your choice.

2 Report

There is no strict template or page limit for the report, but it must follow the overall structure of a research paper. A common structure would be:

- A title and your name
- A short abstract (one paragraph)
- Introduction
- Related Work
- Methodology
- Results
- Discussion
- Conclusion
- Acknowledgements
- References

Images and tables should have numbers and captions, as in a research paper (Figure 1, Table 1, etc).

You are advised (but not required) to use LaTeX to write your report. Overall, learning LaTeX would be helpful for your career. The following pages might be useful:

- <https://www.latex-tutorial.com/>
- <http://mirrors.rit.edu/CTAN/info/lshort/english/lshort.pdf>

2.1 Experimental Evaluation

Please perform a detailed experimental analysis in your report. For instance, you can study the impact of: (i) your own variations of the algorithm; (ii) hyper-parameters; (iii) problem settings

(environment size, etc); (iv) different environments.

You are strongly recommended to perform a statistical analysis to study your experimental results. At a bare minimum, you should consider multiple runs, so that the mean and the variance can be studied. If that is not possible in your project, then please justify why in the report.

The presentation of your results is also important. Please use plots and tables, as in a research paper. You are advised (but not required) to use tools such as Matplotlib (<https://matplotlib.org/>), or R (<https://www.r-project.org/>) for your plots.

2.2 AI Usage

Regarding your report, AI tools can be used to improve the grammar and readability of your text. However, the content of the text must be your own work. AI usage in the implementation will be discussed in the next section.

3 Implementation

There are many existing codes and tutorials available, as well as AI tools that can generate code. However, working in your own implementation is a great exercise for learning deeply all the details of a particular algorithm. At least your own ideas and variations you should be implementing yourself. Hence, there are three potential ways to approach your implementation work. You can either:

- Start with an existing implementation, and then work on your own variations yourself;
- Use an AI tool to have an initial implementation of the standard algorithm, and then work on your own variations yourself; or
- Fully implement the whole project from scratch.

Other variations might also be possible depending on the project (such as combining multiple existing code, etc). The key point is that there must be some implementation work that was done by yourself. We will value the amount of “human” implementation work and effort taken in your project. See the Statement section below.

3.1 Statement

Please include a file “statement.pdf” with your submission. In this file, please write in detail how you approached the implementation work. That is, you must write in detail:

- Which parts of the code were taken from existing sources.
- Which parts of the code were automatically generated by AI tools.
- Which parts of the code are your own work.

In addition, please also write comments in the code to help easily identify which parts are from existing sources, which parts are AI generated, and which parts constitute your own individual work.

Students will lose points if the statement file is unclear or missing. Additionally, false declarations (such as, e.g., plagiarism) could lead to a zero in the whole assignment.

4 Collaborative Work

This is an individual assignment. Collaborative work will be allowed in the following cases:

- Understand the concepts and algorithms.
- Install and understand how to use specific frameworks.
- Understand how to use text formatting frameworks (e.g., LaTeX), and plotting tools (e.g., Matplotlib, R).
- Discuss and brainstorm potential project ideas.
- Limited and specific implementation help (e.g., a student helps another find a bug in their code).

Apart from that, every student must work individually in their own project, and in their own report.

To exercise good academic practice, please write in your report an acknowledgement in case anyone helped you in your project (in the Acknowledgements section).

5 Point Distribution and Submission

The points (in terms of 100%) will be distributed as follows:

- Quality of the writing and presentation (20%)
- Creativity (20%)
- Implementation quality and effort (30%).
- Experimental evaluation (30%).

Regarding your submission, you must include 3 files:

- **report.pdf**, containing your report. Please submit it as a pdf.
- **code.zip**, a compressed file containing all your source code. Other well-known extensions are also ok (.tar.gz, etc).
- **statement.pdf**, containing your text explaining how the implementation work was performed. Please submit it as a pdf.

6 Interviews

If needed, we may interview students to clarify their work. That is, in principle the grade will be decided based on what was submitted, but if there are academic integrity concerns, such as potential false content in the statement file, duplicated work, plagiarism, etc, then we may interview the specific students to finalise their grades.